

# Clean Architecture Blueprint

---

This blueprint defines the structural skeleton for the **Event Manager** multi-tenant SaaS platform. It evolves from the original repository layout by introducing Clean Architecture layers, hexagonal (ports & adapters) boundaries within each bounded context, and first-class multi-tenancy support.

**Reference:** See the [Architectural Audit Report](file:///C:/Users/Hajar/.gemini/antigravity-ide/brain/9024f939-3693-4a5c-93c1-6cc32aabd9bc/architectural\_audit\_report.md) for full rationale behind every structural change.

## Project Overview

Event Manager is an event management application that helps users create and organize weddings, birthdays, private parties, conferences, seminars, corporate events, and other celebrations. It simplifies digital invitations, RSVP tracking, guest management, seating arrangements, menu selection, and dietary restrictions, all from a single platform.

## Actors & Roles

### Admin (Platform)

- Gérer les comptes et abonnements (facturation SaaS)
- Consulter les statistiques globales de la plateforme
- Gérer les modèles d'invitation et les canaux d'envoi
- Gérer les demandes d'impression hybride
- Suspendre/activer un compte
- Gérer les rôles et permissions

### Organisateur (propriétaire du compte)

- Créer un compte
- Créer un Événement (pour soi-même ou pour un Client)
- Consulter le Tableau de Bord
- Générer un lien d'accès pour le Client
- Accéder à l'espace Événement du Client (pour assistance)

### Organisateur & Clients (accès partagé à l'espace Événement)

- Importer et gérer la liste des invités (recherche, filtres, groupes)
- Choisir un modèle ou personnaliser l'invitation
- Choisir le(s) canal(aux) d'envoi et planifier l'envoi
- Lancer l'envoi des invitations (includes: Consulter la liste des invités)
- Gérer le Plan de Table
- Consulter les rapports menus/allergies
- Demander une impression hybride
- Gérer les modifications de dernière minute/annulations

## L'Invité

- Recevoir l'invitation
- Répondre au RSVP (includes: Choisir le Menu & Déclarer les Allergies)

## Personnel d'Accueil

- Scanner le QR Code à l'arrivée et confirmer la présence

## Project Organization (Tâches Principales)

### 1. Planning, Analyse & Design

- Création du diagramme de cas d'utilisation
- Définition des acteurs du système et des interactions principales
- Création du diagramme de classes
- Définition des entités principales, attributs et relations
- Définition de l'architecture du projet
- Choix du design UI/UX, thème et identité visuelle

### 2. Préparation du Projet & Environnement

- Initialisation du dépôt Git
- Ajout des membres de l'équipe et configuration des accès
- Préparation de la structure du projet et configuration initiale
- Configuration des environnements Dev, Test et Production

### 3. Environnement de Démonstration & Suivi d'Avancement

- Préparation du serveur de démonstration
- Configuration des accès serveur et paramètres de déploiement
- Préparation de la configuration Docker
- Déploiement de l'application sur le serveur de démonstration
- Mise en place des mises à jour pour les démonstrations d'avancement
- Validation de la disponibilité et de la stabilité de l'application

### 4. CI/CD & Workflow de Développement

- Mise en place du pipeline CI/CD
- Configuration du build et du déploiement automatique
- Définition de la gestion des versions et du processus de release

### 5. Développement MVP

- Début du développement des fonctionnalités principales
- Implémentation des modèles de données selon le diagramme de classes
- Développement des workflows métier principaux
- Intégration des services et composants nécessaires
- Tests continus et validation des fonctionnalités

## 6. Release MVP

- Préparation de la première version MVP
  - Déploiement de la version MVP sur l'environnement de démonstration
  - Validation des fonctionnalités implémentées
  - Collecte des retours et préparation du backlog d'amélioration
- 

## Frameworks And Core Dependencies

### Backend

- Framework: Spring Boot 3.5.7
- Language: Java 21
- Persistence: Spring Data JPA, Flyway, H2 (dev), MySQL Connector (prod)
- Web: Spring Web, Spring Validation, Spring MVC
- Security: Spring Security, JWT (`jjwt-api`, `jjwt-impl`, `jjwt-jackson`)
- Documentation: Springdoc OpenAPI, Spring REST Docs
- Observability: Spring Actuator, Micrometer (metrics), OpenTelemetry (tracing)
- Messaging: Spring Application Events (in-process), Spring AMQP / Kafka (evolution path)
- Caching: Spring Cache + Caffeine (local) / Redis (distributed)
- Scheduling: Spring `@Scheduled`, Spring Task Executor
- Support: Lombok, Spring AOP, Spring Mail, MapStruct (mapping)

### Frontend

- Framework: Angular 21
- Runtime: TypeScript 5.9, RxJS 7.8
- **UI foundation: OneUI 5.12** (Bootstrap 5.3.8 admin dashboard template by pixelcave)
- OneUI bundled plugins (used directly, **do not replace with alternatives** unless noted):
  - Layout & navigation: Bootstrap 5, Popper.js, SimpleBar (custom scrollbars)
  - **Data tables: AG Grid** (replaces OneUI's DataTables.net — AG Grid is the mandatory default for all tables)
  - Charts: Chart.js, Easy Pie Chart
  - Calendar: FullCalendar
  - Forms: Flatpickr (date/time), ng-select (replaces Select2), Bootstrap Datepicker, Bootstrap Maxlength, Dropzone (file upload), CropperJS (image cropping)
  - Rich text: CKEditor 5 (classic + inline)
  - Notifications: SweetAlert2, Bootstrap Notify
  - Media: Magnific Popup (lightbox), Slick Carousel
  - Maps: jVectorMap
  - Misc: Ion RangeSlider, Raty.js (ratings), Highlight.js, SimpleMDE (markdown)
  - Icons: Font Awesome 7
  - Export: jsPDF (via pdfmake), JSZip
- i18n: `@ngx-translate/core`, `@ngx-translate/http-loader`
- State management: Angular Signals (built-in) for component state; NgRx (optional, complex features)
- Tooling: Angular CLI, Angular build tooling, Vitest, jsdom

**⚠ CRITICAL RULE — NO CUSTOM UI CREATION**

The frontend **must** use OneUI 5.12 pages, layouts, blocks, and components as the sole UI foundation. **Never** create new UI from scratch. All screens must be built by adapting existing OneUI HTML templates into Angular standalone components. The OneUI SCSS theme, variables, and design system must be preserved. Any UI component not available in OneUI should be requested as a OneUI extension, not reinvented.

OneUI 5.12 Integration Strategy

OneUI is an HTML/SCSS/JS template. Integration into Angular follows this approach:

- 1. **SCSS integration:** Import OneUI's `_scss/main.scss` into Angular's `styles.scss`. Use OneUI's SCSS variables (`_variables.scss`, `_variables-bootstrap.scss`, `_variables-themes.scss`) for all custom styling. Never override OneUI's design tokens — extend via `_scss/custom/`.
- 2. **Layout wrapping:** Adapt OneUI's HTML layout shells (`be_layout_*`, `gs_backend.html`) into Angular layout components (`layout/dashboard-layout/`, `layout/public-layout/`). The sidebar, header, and side-overlay structures come directly from OneUI.
- 3. **Page templates:** Each feature page is built by adapting an existing OneUI page template (e.g., `be_pages_dashboard.html` → `features/dashboard/`, `be_pages_ecom_orders.html` → `features/guests/guest-list/`, `be_pages_generic_invoice.html` → `features/billing/`).
- 4. **JS plugins:** OneUI's bundled JS plugins (Chart.js, Flatpickr, SweetAlert2, etc.) are consumed via Angular wrappers or direct initialization in `AfterViewInit` lifecycle hooks. **AG Grid replaces DataTables.net** for all table/grid needs — use `ag-grid-angular` directly. **jQuery is NOT used** — jQuery-dependent plugins (DataTables, Select2, jQuery Sparkline) are replaced with Angular-native alternatives (AG Grid, ng-select, Chart.js).
- 5. **Blocks system:** Use OneUI's block system (`be_blocks_*`) for all content panels. Blocks provide built-in loading states, fullscreen, pin, close, and refresh capabilities.
- 6. **Authentication pages:** Use OneUI's `op_auth_signin*.html`, `op_auth_signup*.html`, `op_auth_reminder*.html` templates for login, registration, and password reset.
- 7. **Error pages:** Use OneUI's `op_error_*.html` templates for 400, 401, 403, 404, 500, 503 error pages.
- 8. **Dark mode:** OneUI has built-in dark mode support. Use it via the layout API, not a custom implementation.
- 9. **RTL support:** OneUI has built-in RTL support (`_rtl-support.scss`, `gs rtl_*`). Use it for Arabic/Hebrew locales.

OneUI Page Template → Feature Mapping

| OneUI Template                                                                 | Maps To Feature                            | Usage                                |
|--------------------------------------------------------------------------------|--------------------------------------------|--------------------------------------|
| <code>be_pages_dashboard.html</code> / <code>be_pages_dashboard_v1.html</code> | <code>features/dashboard/</code>           | Organizer dashboard                  |
| <code>be_tables_datatables.html</code> (layout only, use AG Grid)              | <code>features/guests/guest-list/</code>   | Guest list with search, filter, sort |
| <code>be_forms_elements.html</code> / <code>be_forms_layouts.html</code>       | <code>features/events/event-create/</code> | Event creation form                  |

| OneUI Template                                       | Maps To Feature                         | Usage                            |
|------------------------------------------------------|-----------------------------------------|----------------------------------|
| be_comp_calendar.html                                | features/events/event-detail/           | Event timeline / calendar view   |
| be_pages_generic_inbox.html                          | features/invitations/                   | Invitation management list       |
| be_pages_blog_story.html                             | features/invitations/invitation-editor/ | Rich invitation content editor   |
| be_pages_ecom_orders.html (layout only, use AG Grid) | features/rsvp/rsvp-tracking/            | RSVP response tracking table     |
| be_comp_charts.html                                  | features/menus/                         | Menu/allergy statistical reports |
| be_pages_ecom_dashboard.html                         | features/admin/platform-stats/          | Platform-wide analytics          |
| be_pages_generic_pricing_plans.html                  | features/billing/plan-selection/        | Subscription plan picker         |
| be_pages_generic_invoice.html                        | features/billing/billing-history/       | Invoice detail view              |
| be_pages_generic_profile.html                        | features/admin/tenant-management/       | Tenant profile / detail          |
| be_pages_generic_team.html                           | features/admin/user-management/         | Team / user list management      |
| be_pages_generic_search.html                         | shared/components/global-search/        | Global search results            |
| be_pages_auth_all.html /<br>op_auth_signin*.html     | core/auth/login/                        | Authentication pages             |
| be_pages_error_all.html /<br>op_error_*.html         | core/error-handling/                    | Error pages                      |
| be_pages_generic_contact.html                        | features/print/                         | Hybrid print request form        |
| be_pages_generic_blank_block.html                    | Scaffold template                       | Starting point for new pages     |
| be_widgets_stats.html /<br>be_widgets_tiles.html     | shared/components/                      | Reusable stat/tile widgets       |

| OneUI Template             | Maps To Feature         | Usage                   |
|----------------------------|-------------------------|-------------------------|
| be_comp_notifications.html | features/notifications/ | Notification center     |
| op_checkout.html           | features/billing/       | Checkout / payment flow |

New Dependencies (vs. original)

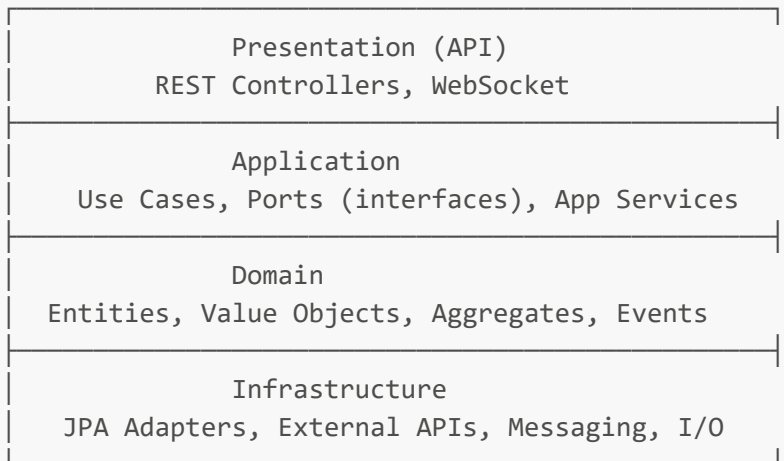
| Dependency                  | Purpose                                                     | Justification                                                  |
|-----------------------------|-------------------------------------------------------------|----------------------------------------------------------------|
| OneUI 5.12                  | <b>Mandatory</b> UI foundation (Bootstrap 5 admin template) | All frontend UI built on this                                  |
| AG Grid (Community)         | <b>Mandatory</b> default table/grid component               | Replaces DataTables.net — no jQuery dependency, Angular-native |
| ng-select                   | Angular-native select component                             | Replaces Select2 — no jQuery dependency                        |
| Micrometer                  | Metrics export (Prometheus, CloudWatch)                     | Observability gap                                              |
| OpenTelemetry               | Distributed tracing                                         | Cross-service tracing                                          |
| Caffeine / Redis            | Caching                                                     | No caching existed                                             |
| MapStruct                   | Bean mapping (domain ↔ persistence ↔ DTO)                   | Replaces manual mappers                                        |
| Spring AMQP or Kafka client | Async messaging (future)                                    | Domain event evolution path                                    |

Starter Notes

- Keep backend and frontend dependencies separated by project boundary.
- Multi-tenancy infrastructure is now a first-class architectural concern, not an afterthought.
- Preserve standalone Angular composition; do not reintroduce modules.
- **All UI must be built using OneUI 5.12 templates and components. Never create custom UI from scratch.**
- OneUI's SCSS variables and design system are the single source of truth for styling.
- **jQuery is NOT permitted.** All jQuery-based OneUI plugins (DataTables, Select2, jQuery Sparkline, etc.) are replaced with Angular-native alternatives (AG Grid, ng-select, Chart.js).
- AG Grid is the **mandatory default** for all data tables and grids.
- Introduce new dependencies only when they close a structural gap and are not already covered by OneUI or the approved replacements.

Architectural Principles

Clean Architecture Layers



**Dependency rule:** Dependencies point inward. Domain has zero framework imports. Application defines ports; Infrastructure implements them.

## Hexagonal Architecture (Ports & Adapters)

Each bounded context follows this internal structure:

```

context-name/
├── domain/           ← Pure business logic (no framework deps)
│   ├── model/       ← Entities, Aggregate Roots, Value Objects
│   ├── event/       ← Domain events
│   ├── service/     ← Domain services (stateless business rules)
│   └── exception/   ← Domain-specific exceptions
├── application/     ← Use cases & orchestration
│   ├── port/
│   │   ├── in/      ← Driving ports (use case interfaces)
│   │   └── out/     ← Driven ports (repository, messaging, storage interfaces)
│   ├── service/     ← Application services implementing driving ports
│   └── dto/         ← Application-level DTOs (command/query objects)
├── infrastructure/ ← Framework-dependent implementations
│   ├── persistence/
│   │   ├── entity/  ← JPA entities (persistence models)
│   │   ├── mapper/  ← JPA entity ↔ domain model mappers
│   │   ├── repository/ ← Spring Data JPA repositories
│   │   └── spec/    ← JPA Specifications for dynamic queries
│   ├── messaging/   ← Event publisher/subscriber adapters
│   ├── external/    ← External API clients
│   └── config/      ← Context-specific Spring configuration
└── presentation/   ← API surface
    ├── resource/    ← REST controllers
    ├── dto/         ← Request/Response DTOs
    └── mapper/      ← Presentation ↔ Application DTO mappers
  
```

**Note:** This replaces the original flat `entity/service/repository/resource/dto/` layout. The original convention names are preserved within the appropriate layer (e.g., `resource/` stays as the

controller package name).

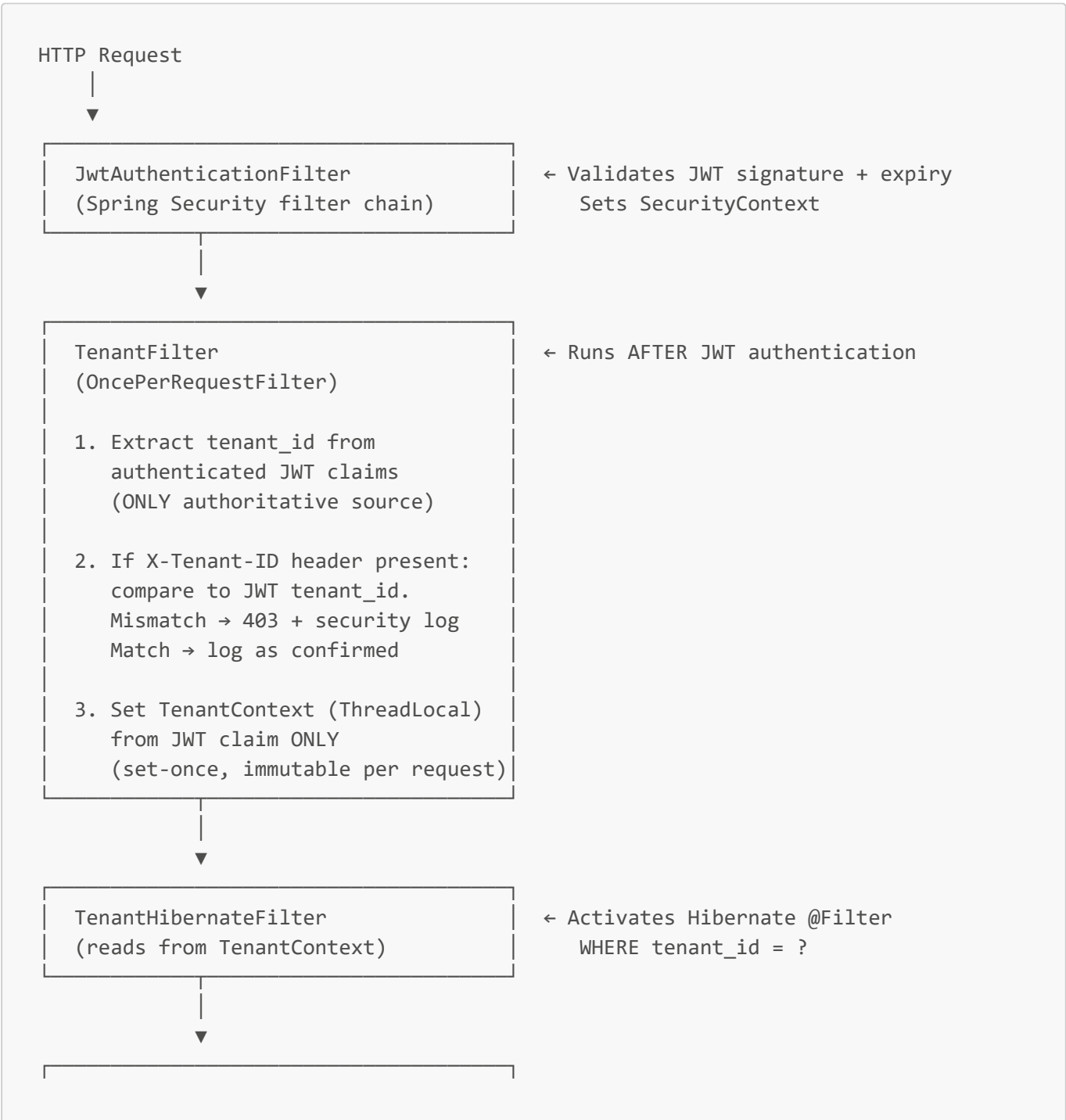
## Multi-Tenancy Architecture

Strategy: Shared Database, Discriminator Column (ADR-002)

All tenant-scoped entities include a `tenant_id` column. Data isolation is enforced at the ORM level via Hibernate `@Filter` + `@FilterDef`, activated automatically by a servlet filter on every request.

### Tenant Context Propagation Pipeline

See [Tenant Filter Security Redesign](file:///C:/Users/Hajar/.gemini/antigravity-ide/brain/9024f939-3693-4a5c-93c1-6cc32aabd9bc/tenant\_filter\_security\_redesign.md) for full implementation details, threat model, and test suite.





|          |                                |
|----------|--------------------------------|
| Database | ← Row-level isolation enforced |
|----------|--------------------------------|

Tenant source trust hierarchy (ADR-002-A):

| Source                               | Trust Level                                       | Used For                                                     |
|--------------------------------------|---------------------------------------------------|--------------------------------------------------------------|
| JWT <code>tenant_id</code> claim     | <b>Authoritative</b> — cryptographically verified | Sets <code>TenantContext</code> , activates Hibernate filter |
| <code>X-Tenant-ID</code> HTTP header | <b>Advisory</b> — client-controlled, untrusted    | Logging, mismatch detection (triggers 403 on conflict)       |
| Subdomain (Host header)              | <b>Untrusted at request time</b>                  | Token issuance only, never at request time                   |

Tenant Package Structure

|                                                |                                                            |
|------------------------------------------------|------------------------------------------------------------|
| tenant/                                        |                                                            |
| ├─ domain/                                     |                                                            |
| │   ├─ model/                                  |                                                            |
| │   │   ├─ Tenant.java                         | ← Tenant aggregate root                                    |
| │   │   ├─ TenantId.java                       | ← Value object                                             |
| │   │   ├─ TenantStatus.java                   | ← ACTIVE, SUSPENDED, TRIAL, CLOSED                         |
| │   │   └─ TenantConfiguration.java            | ← Per-tenant settings (branding, limits)                   |
| │   └─ event/                                  |                                                            |
| │       ├─ TenantCreatedEvent.java             |                                                            |
| │       ├─ TenantSuspendedEvent.java           |                                                            |
| │       └─ TenantPlanChangedEvent.java         |                                                            |
| ├─ application/                                |                                                            |
| │   ├─ port/                                   |                                                            |
| │   │   ├─ in/                                 |                                                            |
| │   │   │   ├─ CreateTenantUseCase.java        |                                                            |
| │   │   │   ├─ SuspendTenantUseCase.java       |                                                            |
| │   │   │   └─ ResolveTenantUseCase.java       |                                                            |
| │   │   └─ out/                                |                                                            |
| │   │       └─ TenantRepositoryPort.java       |                                                            |
| │   └─ service/                                |                                                            |
| │       └─ TenantApplicationService.java       |                                                            |
| ├─ infrastructure/                             |                                                            |
| │   ├─ persistence/                            |                                                            |
| │   │   ├─ entity/TenantJpaEntity.java         |                                                            |
| │   │   ├─ mapper/TenantPersistenceMapper.java |                                                            |
| │   │   └─ repository/TenantJpaRepository.java |                                                            |
| │   └─ context/                                |                                                            |
| │       ├─ TenantContext.java                  | ← ThreadLocal tenant holder (set-once immutable)           |
| │       └─ TenantFilter.java                   | ← OncePerRequestFilter: JWT-only trust, mismatch detection |
| │       └─ TenantHibernateFilter.java          | ← Activates Hibernate @Filter                              |
| │       └─ TenantAwareTaskDecorator.java       | ← [NEW] Propagates TenantContext to                        |

```

@Async threads (ADR-008)
| |   └─ TenantAwareAuthenticationDetails.java ← Carries tenantId for non-JWT
auth
|   └─ config/
|       └─ TenantFilterConfig.java
|   └─ test/
|       └─ TenantFilterSecurityTest.java ← 8 security tests: spoofing,
immutability, JWT-only
└─ presentation/
    ├── resource/TenantResource.java    ← Admin-only tenant management API
    └─ dto/

```

## Base Entity Hierarchy

```

// shared-kernel/domain/model/

// Domain layer – plain Java, no framework annotations
public abstract class BaseEntity {
    private Long id;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private String createdBy;
    private String updatedBy;
}

public abstract class TenantScopedEntity extends BaseEntity {
    private Long tenantId; // Discriminator column – auto-set by TenantContext
}

// Soft delete support
public abstract class SoftDeletableEntity extends TenantScopedEntity {
    private boolean deleted = false;
    private LocalDateTime deletedAt;
    private String deletedBy;
}

```

**Important:** The JPA annotations (`@MappedSuperclass`, `@Entity`, etc.) belong on the **infrastructure persistence models only**, not on these domain base classes. The mapping between domain and persistence models is handled by the persistence mapper in each bounded context.

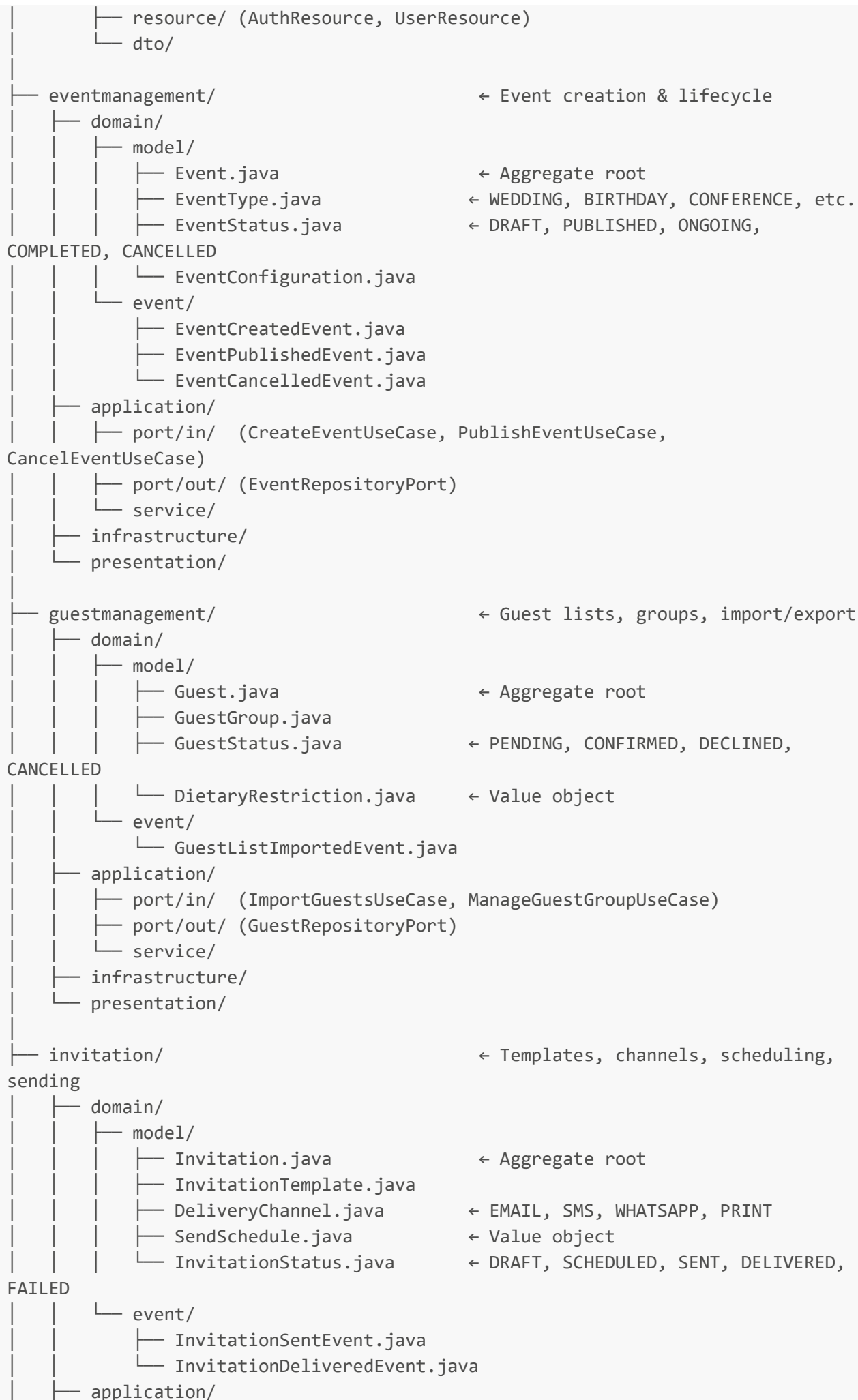
## Backend Architecture — Spring Boot (Updated)

```

com.eventmanager/
├─ Application.java
├─ sharedkernel/                                ← Cross-context shared domain
│   ├── domain/
│   │   └─ model/

```



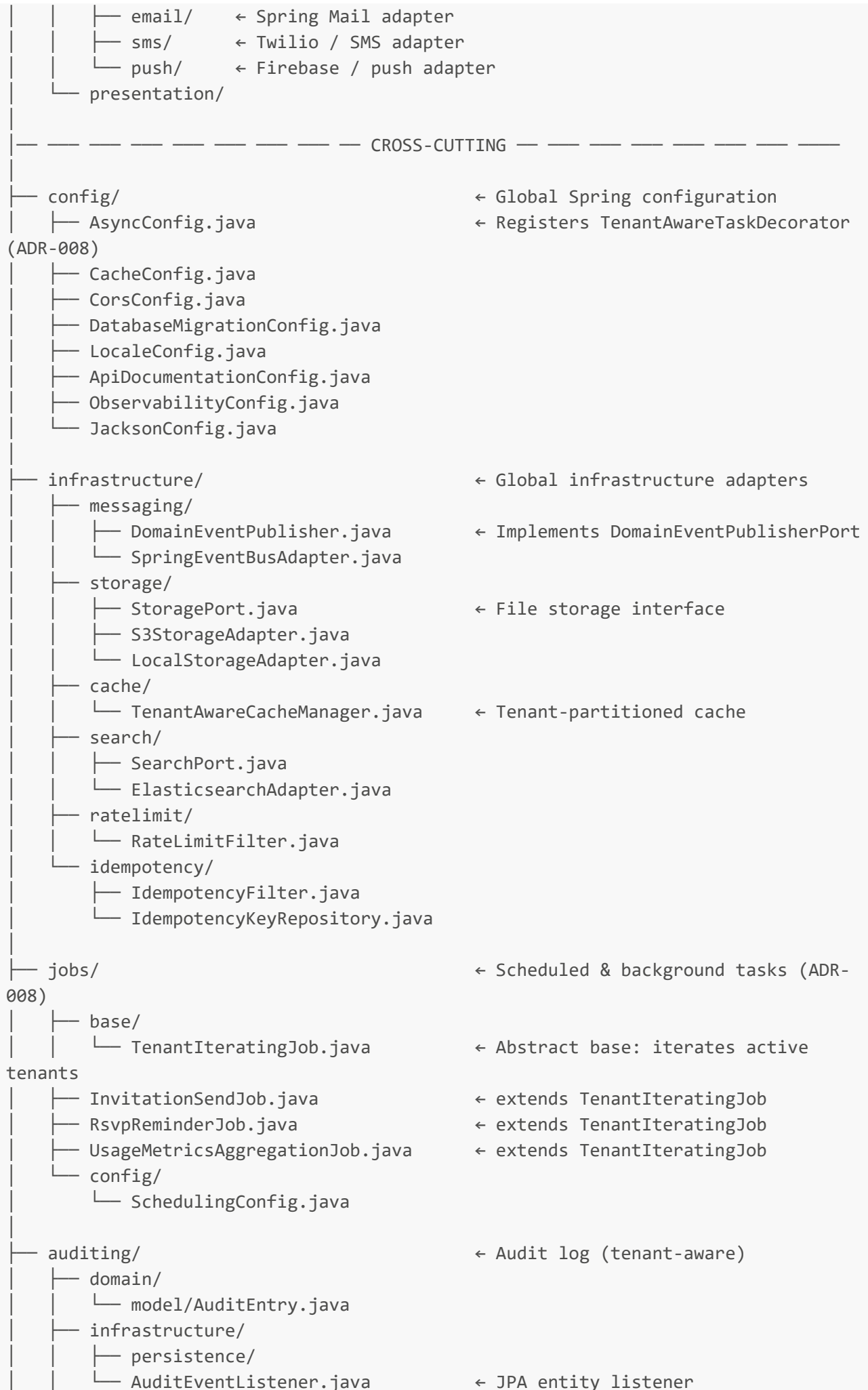


```

|   |   | — port/in/ (CreateInvitationUseCase, ScheduleSendUseCase,
|   |   | SendInvitationsUseCase)
|   |   | — port/out/ (InvitationRepositoryPort, DeliveryChannelPort,
|   |   | TemplateStoragePort)
|   |   | — service/
|   |   | — infrastructure/
|   |   | — persistence/
|   |   | — delivery/ ← Email, SMS, WhatsApp adapter
implementations
|   |   | — template/ ← Template storage adapter
|   |   | — presentation/
| — rsvp/ ← Responses, menu choices, dietary
declarations
|   | — domain/
|   |   | — model/
|   |   |   | — RsvpResponse.java ← Aggregate root
|   |   |   | — MenuChoice.java
|   |   |   | — AllergyDeclaration.java
|   |   |   | — RsvpStatus.java ← PENDING, ACCEPTED, DECLINED
|   |   | — event/
|   |   |   | — RsvpReceivedEvent.java
|   | — application/
|   |   | — port/in/ (SubmitRsvpUseCase, UpdateMenuChoiceUseCase)
|   |   | — port/out/ (RsvpRepositoryPort)
|   |   | — service/
|   | — infrastructure/
|   | — presentation/
| — seating/ ← Table plans, assignments
|   | — domain/
|   |   | — model/
|   |   |   | — SeatingPlan.java ← Aggregate root
|   |   |   | — Table.java
|   |   |   | — SeatAssignment.java
|   |   | — event/
|   |   |   | — SeatingPlanFinalizedEvent.java
|   | — application/
|   | — infrastructure/
|   | — presentation/
| — checkin/ ← QR code generation, arrival
scanning
|   | — domain/
|   |   | — model/
|   |   |   | — CheckIn.java ← Aggregate root
|   |   |   | — QrCode.java ← Value object
|   |   | — event/
|   |   |   | — GuestCheckedInEvent.java
|   | — application/
|   |   | — port/in/ (GenerateQrCodeUseCase, ScanQrCodeUseCase)
|   |   | — port/out/ (CheckInRepositoryPort, QrCodeGeneratorPort)
|   |   | — service/
|   | — infrastructure/

```

14 / 29



```

├── presentation/
│   └── resource/AuditResource.java
├── initialization/                                ← Data seeding / bootstrap
│   ├── initializers/
│   └── DataInitializationCoordinator.java

```

## Package Naming Convention

- All Java package names use **lowercase concatenated words** — no hyphens, no underscores.
- Example: `com.eventmanager.sharedkernel.domain.model`, NOT `com.eventmanager.shared-kernel.domain.model`.
- Frontend (Angular/TypeScript) folders continue to use **kebab-case** as per Angular convention.

## Backend Layer Rules

| Layer                        | May Import                               | Must NOT Import                        |
|------------------------------|------------------------------------------|----------------------------------------|
| <code>domain/</code>         | Java stdlib, sharedkernel domain         | Spring, JPA, any framework             |
| <code>application/</code>    | domain, sharedkernel                     | Infrastructure, presentation           |
| <code>infrastructure/</code> | domain, application (ports), Spring, JPA | presentation                           |
| <code>presentation/</code>   | application (ports/DTOs), Spring Web     | domain models directly, infrastructure |

## Backend Linkage Notes (Preserved + Extended)

- `presentation/resource/` exposes HTTP endpoints — replaces the original `resource/` convention.
- `application/service/` contains use case orchestration — replaces the original `service/` convention.
- `infrastructure/persistence/repository/` handles persistence access — replaces the original `repository/`.
- `domain/model/` holds pure domain entities — replaces the original `entity/` (which mixed domain and JPA).
- `application/dto/` and `presentation/dto/` isolate command/query and request/response payloads.
- `infrastructure/persistence/mapper/` converts between domain models and JPA entities.
- `presentation/mapper/` converts between presentation DTOs and application DTOs.
- `identity/infrastructure/security/` groups JWT, API key, filter, and authentication support.
- `config/` centralizes framework configuration such as CORS, Flyway, locale, async, caching, and API docs.
- `config/AsyncConfig` registers `TenantAwareTaskDecorator` for tenant context propagation to `@Async` threads (ADR-008).
- `jobs/base/TenantIteratingJob` is the base class for all scheduled jobs — iterates active tenants from the database (ADR-008).
- `subscription/infrastructure/entitlement/` provides `@RequiresEntitlement` annotation and AOP interceptor for plan-based feature gating (ADR-009).

## Cross-Cutting Backend Concerns



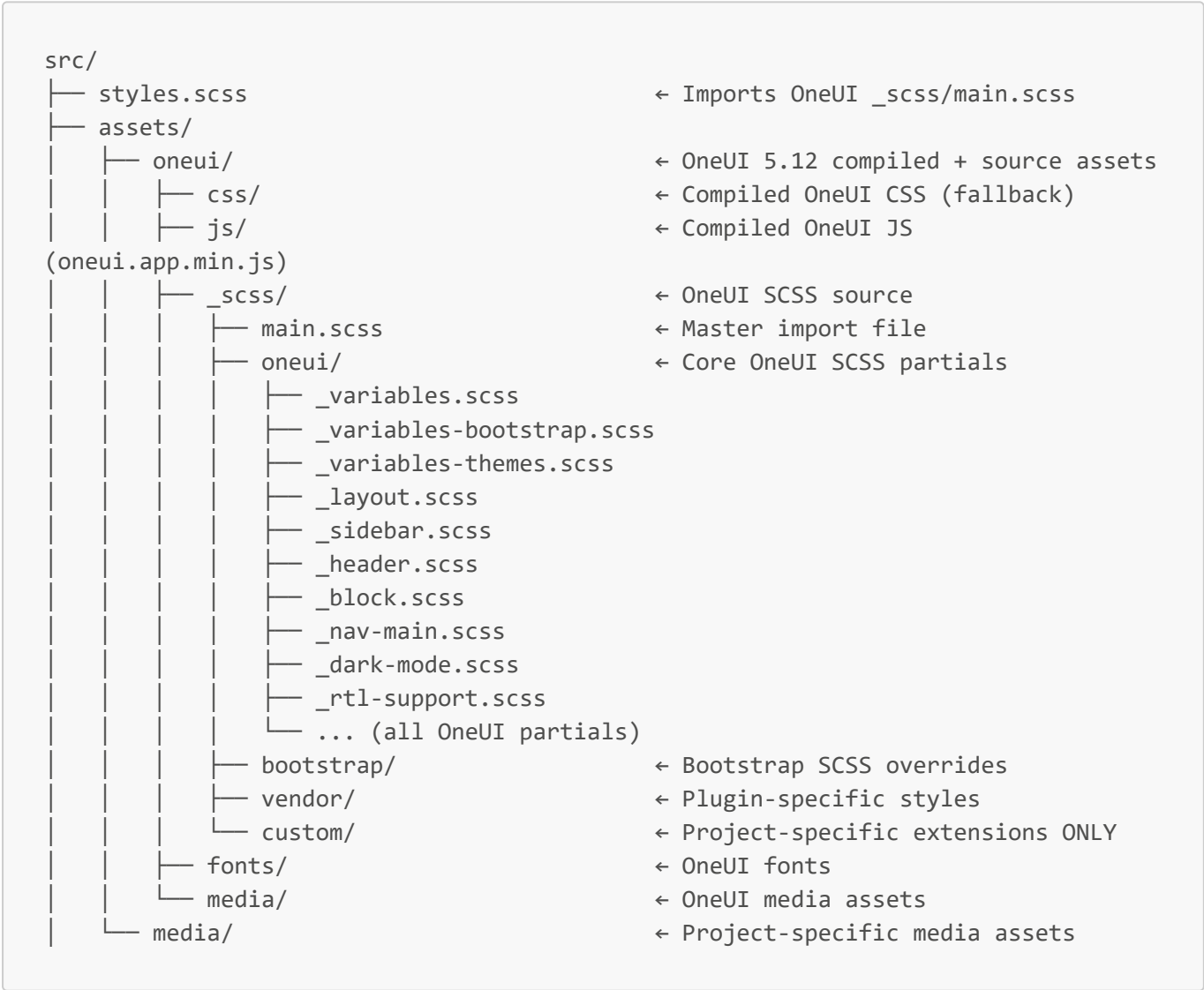
| Concern                         | Location                                                               | Mechanism                                                                                                     |
|---------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Authentication                  | <code>identity/infrastructure/security/</code>                         | JWT filter + Spring Security filter chain                                                                     |
| Authorization                   | <code>identity/</code> + per-context <code>@PreAuthorize</code>        | RBAC with method-level security                                                                               |
| Multi-tenancy                   | <code>tenant/infrastructure/context/</code>                            | <code>TenantFilter</code> → <code>TenantContext</code> → Hibernate <code>@Filter</code> (ADR-002-A: JWT-only) |
| Tenant propagation (@Async)     | <code>tenant/infrastructure/context/</code>                            | <code>TenantAwareTaskDecorator</code> via <code>AsyncConfig</code> (ADR-008)                                  |
| Tenant propagation (@Scheduled) | <code>jobs/base/</code>                                                | <code>TenantIteratingJob</code> — iterates tenants from DB (ADR-008)                                          |
| Feature gating                  | <code>subscription/infrastructure/entitlement/</code>                  | <code>@RequiresEntitlement</code> + AOP interceptor (ADR-009)                                                 |
| Audit logging                   | <code>auditing/</code>                                                 | JPA entity listener + <code>TenantContext</code>                                                              |
| Exception handling              | <code>config/</code> + <code>sharedkernel/domain/exception/</code>     | <code>@ControllerAdvice</code> global handler                                                                 |
| Validation                      | <code>sharedkernel/application/validation/</code>                      | Bean Validation + self-validating commands                                                                    |
| Localization (i18n)             | <code>config/LocaleConfig</code> + message bundles                     | <code>MessageSource</code> with <code>Accept-Language</code> header                                           |
| API versioning                  | URL path prefix <code>/api/v1/</code>                                  | Defined in controller <code>@RequestMapping</code>                                                            |
| Rate limiting                   | <code>infrastructure/ratelimit/</code>                                 | Servlet filter with configurable per-tenant limits                                                            |
| Idempotency                     | <code>infrastructure/idempotency/</code>                               | Idempotency-Key header filter                                                                                 |
| Caching                         | <code>infrastructure/cache/</code> + <code>config/CacheConfig</code>   | Spring Cache + tenant-partitioned keys                                                                        |
| Soft delete                     | <code>sharedkernel/domain/model/SoftDeletableEntity</code>             | <code>@SQLRestriction("deleted = false")</code>                                                               |
| Pagination                      | Standardized <code>PageRequest</code> / <code>PageResponse</code> DTOs | Consistent across all list endpoints                                                                          |

| Concern         | Location                                | Mechanism                                                                                    |
|-----------------|-----------------------------------------|----------------------------------------------------------------------------------------------|
| Observability   | <code>config/ObservabilityConfig</code> | Micrometer metrics +<br>OpenTelemetry tracing                                                |
| Background jobs | <code>jobs/</code>                      | <code>TenantIteratingJob</code> +<br>Spring <code>@Scheduled</code> + async<br>task executor |

## Frontend Architecture — Angular Standalone + OneUI 5.12

**UI Foundation:** All frontend pages, layouts, blocks, and components are built exclusively from [OneUI 5.12](file:///c:/Users/Hajar/OneDrive/Documents/EventManager/OneUI%205.12) templates. No custom UI is created from scratch.

### OneUI Asset Integration



|                                  |                                       |
|----------------------------------|---------------------------------------|
| src/app/                         |                                       |
| ├─ app.config.ts                 | ← Providers bootstrap (incl. OneUI    |
| init)                            |                                       |
| ├─ app.routes.ts                 | ← Route composition + lazy loading    |
| ├─ app.ts                        | ← Root component                      |
| ├─ core/                         |                                       |
| concerns                         | ← Singleton services, app-wide        |
| ├─ auth/                         |                                       |
| │ ├─ guards/                     |                                       |
| │ │ ├─ auth.guard.ts             |                                       |
| │ │ └─ role.guard.ts             |                                       |
| │ └─ interceptors/               |                                       |
| │ │ └─ auth.interceptor.ts       |                                       |
| ├─ services/                     |                                       |
| │ └─ auth.service.ts             |                                       |
| ├─ login/                        | ← Adapts OneUI op_auth_signin*.html   |
| └─ i18n/                         |                                       |
| ├─ tenant/                       | ← [NEW] Tenant context                |
| │ ├─ tenant.resolver.ts          | ← Resolves tenant from URL/subdomain  |
| │ ├─ tenant.service.ts           | ← Current tenant state + branding     |
| │ ├─ tenant.guard.ts             | ← Ensures valid tenant context        |
| │ └─ tenant.interceptor.ts       | ← Adds X-Tenant-ID header to requests |
| ├─ permissions/                  | ← [NEW] RBAC in UI                    |
| │ ├─ permission.service.ts       |                                       |
| │ └─ has-permission.directive.ts | ← *hasPermission="'MANAGE_GUESTS'"    |
| ├─ error-handling/               | ← [NEW] Adapts OneUI op_error_*.html  |
| │ ├─ global-error-handler.ts     |                                       |
| │ └─ error.interceptor.ts        |                                       |
| ├─ oneui/                        | ← [NEW] OneUI Angular bridge          |
| │ └─ oneui.service.ts            | ← OneUI JS API wrapper (layout        |
| toggle, dark mode, etc.)         |                                       |
| ├─ oneui-block.directive.ts      | ← Angular directive for OneUI block   |
| system                           |                                       |
| ├─ oneui-init.provider.ts        | ← APP_INITIALIZER for OneUI bootstrap |
| ├─ interceptors/                 |                                       |
| │ ├─ error.interceptor.ts        |                                       |
| │ └─ loading.interceptor.ts      |                                       |
| ├─ services/                     |                                       |
| │ └─ api.service.ts              | ← Base HTTP service with /api/v1/     |
| prefix                           |                                       |
| ├─ notification.service.ts       | ← Wraps SweetAlert2 + Bootstrap       |
| Notify                           |                                       |
| ├─ features/                     | ← Domain-aligned feature folders      |
| ├─ dashboard/                    | ← Adapts be_pages_dashboard.html      |
| │ ├─ components/                 |                                       |
| │ │ ├─ dashboard-page/           | ← OneUI dashboard widgets + Chart.js  |
| │ │ └─ stat-widgets/             | ← Adapts be_widgets_stats.html        |
| │ └─ services/                   |                                       |
| │ └─ dashboard.routes.ts         |                                       |
| └─ events/                       | ← Event creation & management         |

|                        |                                      |
|------------------------|--------------------------------------|
| components/            |                                      |
| event-list/            | ← AG Grid table (layout from OneUI)  |
| event-create/          | ← Adapts be_forms_layouts.html       |
| event-detail/          | ← Adapts be_comp_calendar.html       |
| services/              |                                      |
| models/                |                                      |
| events.routes.ts       |                                      |
| guests/                | ← Guest list management              |
| components/            |                                      |
| guest-list/            | ← AG Grid table (layout from OneUI)  |
| guest-import/          | ← Adapts Dropzone file upload        |
| guest-groups/          | ← Adapts be_ui_tabs.html + AG Grid   |
| services/              |                                      |
| models/                |                                      |
| guests.routes.ts       |                                      |
| invitations/           | ← Invitation builder & sending       |
| components/            |                                      |
| template-picker/       | ← Adapts be_comp_gallery.html        |
| invitation-editor/     | ← Adapts CKEditor 5 +                |
| be_forms_editors.html  |                                      |
| send-scheduler/        | ← Adapts Flatpickr +                 |
| be_forms_plugins.html  |                                      |
| services/              |                                      |
| models/                |                                      |
| invitations.routes.ts  |                                      |
| rsvp/                  | ← RSVP public page + tracking        |
| components/            |                                      |
| rsvp-form/             | ← Public: adapts                     |
| be_forms_elements.html |                                      |
| rsvp-tracking/         | ← AG Grid table (layout from OneUI)  |
| menu-selection/        | ← Adapts ng-select +                 |
| be_forms_plugins.html  |                                      |
| services/              |                                      |
| rsvp.routes.ts         |                                      |
| seating/               | ← Seating plan editor                |
| components/            |                                      |
| seating-editor/        | ← Canvas-based (extends OneUI block  |
| container)             |                                      |
| seating-report/        | ← Adapts be_comp_charts.html         |
| services/              |                                      |
| seating.routes.ts      |                                      |
| menus/                 | ← Menu & dietary reports             |
| components/            | ← Adapts be_comp_charts.html +       |
| be_tables_styles.html  |                                      |
| services/              |                                      |
| menus.routes.ts        |                                      |
| check-in/              | ← QR code & arrival scanning         |
| components/            |                                      |
| qr-scanner/            | ← Camera input (extends OneUI block) |
| attendance-list/       | ← AG Grid table (layout from OneUI)  |
| services/              |                                      |
| check-in.routes.ts     |                                      |
| admin/                 | ← Platform admin panel               |
| components/            |                                      |

|                                     |                                       |
|-------------------------------------|---------------------------------------|
| tenant-management/                  | ← Adapts                              |
| be_pages_generic_profile.html       |                                       |
| user-management/                    | ← Adapts be_pages_generic_team.html   |
| platform-stats/                     | ← Adapts be_pages_ecom_dashboard.html |
| template-management/                | ← Adapts be_comp_gallery.html         |
| services/                           |                                       |
| admin.routes.ts                     |                                       |
| billing/                            | ← Subscription & billing              |
| components/                         |                                       |
| plan-selection/                     | ← Adapts                              |
| be_pages_generic_pricing_plans.html |                                       |
| billing-history/                    | ← Adapts                              |
| be_pages_generic_invoice.html       |                                       |
| usage-dashboard/                    | ← Adapts be_widgets_stats.html +      |
| Chart.js                            |                                       |
| services/                           |                                       |
| billing.routes.ts                   |                                       |
| notifications/                      | ← Adapts be_comp_notifications.html   |
| components/                         |                                       |
| services/                           |                                       |
| notifications.routes.ts             |                                       |
| print/                              | ← Adapts                              |
| be_pages_generic_contact.html       |                                       |
| components/                         |                                       |
| services/                           |                                       |
| print.routes.ts                     |                                       |
| layout/                             | ← OneUI layout shells → Angular       |
| dashboard-layout/                   | ← Adapts gs_backend.html (sidebar +   |
| header + content)                   |                                       |
| public-layout/                      | ← Adapts bd_simple_1.html (guest-     |
| facing)                             |                                       |
| header/                             | ← OneUI header with nav, search,      |
| notifications                       |                                       |
| sidebar/                            | ← OneUI sidebar with nav-main         |
| side-overlay/                       | ← OneUI side overlay panel            |
| shared/                             | ← Reusable building blocks            |
| components/                         |                                       |
| oneui-block/                        | ← Angular wrapper for OneUI block     |
| component                           |                                       |
| data-table/                         | ← AG Grid wrapper (default table for  |
| all features)                       |                                       |
| confirmation-dialog/                | ← Wraps SweetAlert2                   |
| file-upload/                        | ← Wraps Dropzone                      |
| date-picker/                        | ← Wraps Flatpickr                     |
| rich-select/                        | ← Wraps ng-select (replaces Select2)  |
| stat-widget/                        | ← Reusable stat card (OneUI widget)   |
| constants/                          |                                       |
| directives/                         |                                       |
| guards/                             |                                       |
| i18n/                               |                                       |
| models/                             |                                       |
| pagination.model.ts                 | ← Standardized page request/response  |

```
|   └─ api-response.model.ts  
├─ pipes/  
├─ services/  
└─ utils/
```

## Frontend Linkage Notes (Preserved + Extended)

- The app is organized around standalone components and lazy-loaded routes.
- `app.config.ts` bootstraps providers such as HTTP interception, translations, notifications, tenant resolution, OneUI initialization, and app-wide initializers.
- `app.routes.ts` owns route composition and lazy loading. Routes are split by feature with per-feature `.routes.ts` files.
- `core/auth/` holds authentication state, guards, login, and role checks. Login pages adapt OneUI's `op_auth_signin*.html` templates.
- `core/tenant/` **[NEW]** manages tenant context: resolves current tenant, provides branding/config, injects tenant header into API calls.
- `core/permissions/` **[NEW]** provides a `*hasPermission` structural directive for role-based UI rendering.
- `core/error-handling/` **[NEW]** provides global error handler and error pages adapted from OneUI's `op_error_*.html`.
- `core/oneui/` **[NEW]** provides the Angular ↔ OneUI bridge: layout API service, block directive, and app initializer.
- `core/interceptors/` is the place for HTTP cross-cutting concerns such as auth headers, tenant headers, loading states, and error handling.
- `shared/` contains Angular wrappers around OneUI plugins (SweetAlert2, Dropzone, Flatpickr) and Angular-native replacements (AG Grid, ng-select) plus reusable directives, pipes, services, constants, models, and utilities.
- `features/` is domain-oriented and split by bounded context. Each feature adapts specific OneUI page templates (see mapping table above).
- `layout/` contains OneUI layout shells adapted as Angular components: dashboard layout (sidebar + header + content area), public layout (guest-facing), and side overlay.

## OneUI Integration Rules

1. **Never create a custom CSS framework or design system.** OneUI's SCSS is the single source of truth.
2. **Extend, don't override.** Custom styles go in `_scss/custom/` and must use OneUI variables.
3. **Use OneUI blocks for all content panels.** Do not use plain `<div>` containers for page sections.
4. **Use OneUI's built-in responsive breakpoints.** Do not define custom breakpoints.
5. **Use OneUI's color themes.** Do not introduce new color palettes.
6. **Use OneUI's built-in JS API** for layout operations (sidebar toggle, dark mode, side overlay).
7. **jQuery is NOT permitted.** All jQuery-dependent plugins are replaced with Angular-native alternatives (AG Grid, ng-select).
8. **AG Grid is the default table component.** Use `ag-grid-angular` for all data grids. Style it with OneUI's table classes for visual consistency.

## Standalone-Only Guidance (Preserved)

- This blueprint intentionally excludes module-based organization.
- For a new project, keep features as standalone routes and components.
- Place singleton technical services under `core/` and reusable building blocks under `shared/`.
- Keep UI implementation details, CSS, HTML templates, and design assets out of the architectural skeleton when documenting the layout.

---

## API Design Standards

### URL Convention

```
/api/v1/{resource}           ← Collection
/api/v1/{resource}/{id}      ← Single resource
/api/v1/{resource}/{id}/{sub-resource} ← Nested resource
```

All endpoints are implicitly tenant-scoped (tenant ID from JWT, not in URL).

### Standard Response Envelope

```
{
  "success": true,
  "data": { },
  "meta": {
    "page": 0,
    "size": 20,
    "totalElements": 150,
    "totalPages": 8
  },
  "errors": []
}
```

### Pagination & Filtering Convention

- Pagination: `?page=0&size=20&sort=createdAt,desc`
- Filtering: `?status=CONFIRMED&eventType=WEDDING&search=keyword`
- All list endpoints must support pagination by default.

---

## Database & Persistence Strategy

### Migration Tool

Flyway (preserved from original). All migrations in `src/main/resources/db/migration/`.

### Naming Convention

- Tables: `snake_case`, plural (e.g., `events`, `guests`, `invitations`)

- Columns: `snake_case` (e.g., `tenant_id`, `created_at`)
- Indexes: `idx_{table}_{column}` (e.g., `idx_guests_tenant_id`)
- Foreign keys: `fk_{table}_{referenced_table}` (e.g., `fk_guests_events`)

Multi-Tenant Indexes

Every tenant-scoped table must have a composite index on `(tenant_id, id)` and `(tenant_id, {primary_query_column})` to ensure efficient tenant-filtered queries.

Aggregate Boundaries

| Aggregate Root | Owned Entities                   | Rationale                                  |
|----------------|----------------------------------|--------------------------------------------|
| Event          | EventConfiguration               | Event is the central organizing concept    |
| Guest          | DietaryRestriction               | Guest owns their own dietary data          |
| Invitation     | InvitationTemplate, SendSchedule | Invitation controls its delivery lifecycle |
| RsvpResponse   | MenuChoice, AllergyDeclaration   | RSVP is a self-contained response unit     |
| SeatingPlan    | Table, SeatAssignment            | Seating plan is managed as a whole         |
| CheckIn        | QrCode                           | Check-in owns its QR artifact              |
| Tenant         | TenantConfiguration              | Tenant owns its settings                   |
| Subscription   | Plan, BillingCycle, UsageQuota   | Subscription manages billing lifecycle     |
| User           | Role, Permission                 | User is the identity aggregate             |

Testing Strategy

Directory Layout

```
src/test/java/com/eventmanager/  
├── unit/                                ← Domain + application (no Spring context)  
│   ├── event-management/  
│   │   ├── domain/  
│   │   └── application/  
│   └── ...  
├── integration/                        ← With Spring context, real DB (H2)  
│   ├── persistence/  
│   └── api/  
└── e2e/                                ← Full stack tests
```

Test Categories



| Category    | Scope                                             | Speed  | Framework                                     |
|-------------|---------------------------------------------------|--------|-----------------------------------------------|
| Unit        | Domain logic, application services (mocked ports) | Fast   | JUnit 5, Mockito                              |
| Integration | Persistence adapters, API endpoints               | Medium | @SpringBootTest, @DataJpaTest, TestContainers |
| E2E         | Full user flows                                   | Slow   | REST Assured / Playwright                     |

## Architecture Decision Records (ADRs)

### ADR-001: Adopt Hexagonal Architecture (Ports & Adapters) Within Each Bounded Context

**Status:** Accepted **Context:** The original structure uses a flat package-per-feature layout that conflates all layers. **Decision:** Each bounded context contains internal layers: `domain/`, `application/`, `infrastructure/`, `presentation/`. Dependencies point inward. Infrastructure implements ports defined by the application layer. **Consequences:** More packages, but enforceable dependency rules. Domain is testable in isolation.

### ADR-002: Shared Database with Discriminator Column for Multi-Tenancy

**Status:** Accepted **Context:** The platform requires multi-tenancy. Three strategies were evaluated. **Decision:** Use a shared database with a `tenant_id` discriminator column on all tenant-scoped entities. Enforce isolation via Hibernate `@Filter` activated by a `TenantFilter` servlet filter. **Consequences:** Simpler ops, lower cost. Must enforce tenant context at every data access point. Cross-tenant queries require explicit opt-in.

#### ADR-002-A: JWT-Only Tenant Trust Rule (Security Addendum)

**Status:** Accepted **Date:** 2026-07-13 **Supersedes:** The original ADR-002 text which described `tenant_id` resolution from "JWT claim / header / subdomain" without stating precedence or trust hierarchy.

**Threat closed:** Cross-tenant Insecure Direct Object Reference (IDOR) via `X-Tenant-ID` header spoofing. A valid user of Tenant A could set `X-Tenant-ID: <tenant-B-id>` and, if the filter trusted the header, read or write Tenant B's data. CVSS 9.1 (Critical).

**Decision:**

- Single authoritative source.** The tenant ID used for authorization is derived **exclusively** from the `tenant_id` claim inside the validated JWT. The JWT signature guarantees this value was set by the server at token issuance and has not been tampered with.
- X-Tenant-ID header is advisory-only.** If the Angular frontend sends the header, it is used for logging and mismatch detection. It is **never** read into `TenantContext` and **never** used to activate the Hibernate tenant filter.
- Mismatch rejection.** If `X-Tenant-ID` is present and its value differs from the JWT's `tenant_id` claim, the request is rejected with HTTP 403 and a `TENANT_SPOOFING_ATTEMPT` event is written to the security audit log.
- Subdomain resolution** (if used in future) must resolve through a server-side signed mapping — the raw subdomain string from the Host header is never directly trusted as a tenant identifier at request time.

5. **TenantContext immutability.** Once `TenantContext.setTenantId()` is called by `TenantFilter`, it is locked for the remainder of the request. Downstream code cannot mutate the tenant context.
6. **Platform admin bypass.** Requests from users with `ROLE_PLATFORM_ADMIN` authority may operate without a tenant scope (cross-tenant queries). The tenant context remains null for these requests.

Precedence rule (definitive):

| Source                               | Trust Level                                       | Used For                                                     |
|--------------------------------------|---------------------------------------------------|--------------------------------------------------------------|
| JWT <code>tenant_id</code> claim     | <b>Authoritative</b> — cryptographically verified | Sets <code>TenantContext</code> , activates Hibernate filter |
| <code>X-Tenant-ID</code> HTTP header | <b>Advisory</b> — client-controlled, untrusted    | Logging, mismatch detection (triggers 403 on conflict)       |
| Subdomain (Host header)              | <b>Untrusted at request time</b>                  | Token issuance only, never at request time                   |

**Consequences:** No client-controlled input can influence which tenant's data a request accesses. Frontend bugs (stale JWT, tenant switch without re-auth) are detected immediately. JWT must be refreshed when a user switches tenants. Security audit log enables detection of spoofing campaigns.

See [Tenant Filter Security Redesign](file:///C:/Users/Hajar/.gemini/antigravity-ide/brain/9024f939-3693-4a5c-93c1-6cc32aabd9bc/tenant\_filter\_security\_redesign.md) for full implementation (TenantFilter.java, TenantContext.java, tenant.interceptor.ts, TenantFilterSecurityTest.java).

ADR-003: Domain Events for Inter-Context Communication

**Status:** Accepted **Context:** Bounded contexts (Events, Guests, Invitations, RSVP) need to react to each other's state changes without tight coupling. **Decision:** Use Spring Application Events for in-process domain events. Introduce `DomainEvent` as a base type in the shared kernel. Evolve to a message broker (RabbitMQ/Kafka) when scaling beyond a single instance. **Consequences:** Loose coupling between contexts. Eventual consistency must be accepted for cross-context operations.

ADR-004: Separate Domain Entities from Persistence Models

**Status:** Accepted **Context:** JPA-annotated entities in the original `entity/` serve as both domain objects and persistence models. **Decision:** Domain entities (in `domain/model/`) are plain Java objects. JPA entities (in `infrastructure/persistence/entity/`) are mapped via `infrastructure/persistence/mapper/`. **Consequences:** More mapping code, but domain remains framework-independent and testable.

ADR-005: Feature-Aligned Frontend Structure with Named Bounded Contexts

**Status:** Accepted **Context:** The original frontend has 22 placeholder `domain-*` folders with no semantic meaning. **Decision:** Replace with named feature folders aligned to business domains: `events/`, `guests/`, `invitations/`, `rsvp/`, `seating/`, `menus/`, `check-in/`, `dashboard/`, `admin/`, `billing/`. **Consequences:** Self-documenting code. New developers understand the domain from the folder structure.

ADR-006: OneUI 5.12 as Mandatory UI Foundation

**Status:** Accepted **Context:** The project includes a licensed copy of OneUI 5.12, a comprehensive Bootstrap 5 admin dashboard template with 155+ page templates, extensive SCSS theming, dark mode, RTL support, and 25+ bundled JS plugins. **Decision:** All frontend UI must be built exclusively by adapting OneUI 5.12 HTML templates into Angular standalone components. No custom UI is to be created from scratch. OneUI's SCSS design system is the single source of truth for all styling. OneUI's bundled plugins are used directly except where jQuery-dependent plugins are replaced by Angular-native alternatives: **AG Grid** replaces DataTables.net, **ng-select** replaces Select2. jQuery is not used. **Consequences:** Consistent, premium visual quality with zero design effort. Faster development by adapting proven templates. No jQuery dependency — all interactive components are Angular-native, improving testability and change detection integration.

## ADR-007: AG Grid as Default Table Component (No jQuery)

**Status:** Accepted **Context:** OneUI 5.12 bundles DataTables.net which depends on jQuery. jQuery introduces global state, conflicts with Angular's change detection, and increases bundle size. The project needs rich table features (server-side pagination, filtering, sorting, column resizing, row selection, export). **Decision:** Replace DataTables.net with **AG Grid Community** ([ag-grid-angular](#)) as the mandatory default table/grid component. Replace Select2 with **ng-select**. Remove jQuery entirely from the project. Style AG Grid to match OneUI's table aesthetics using custom AG Grid themes that reference OneUI SCSS variables. **Consequences:** Zero jQuery dependency. Full Angular integration (native change detection, typed APIs, tree-shakable). AG Grid Community covers all required table features. Custom theme CSS is needed to match OneUI's visual style.

## ADR-008: Tenant Context Propagation for Async and Scheduled Execution

**Status:** Accepted **Context:** `TenantContext` uses `ThreadLocal`, which does not propagate to `@Async` worker threads or `@Scheduled` jobs. Jobs like `InvitationSendJob` and `RsvpReminderJob` run without an HTTP request or JWT, yet must read/write tenant-scoped data with correct Hibernate filter activation. **Decision:**

1. **@Async tasks** (spawned from HTTP requests): Use a `TenantAwareTaskDecorator` registered in `AsyncConfig` that captures the caller's tenant ID and re-establishes it on the worker thread. The propagated tenant ID was originally verified from a JWT by `TenantFilter`.
2. **@Scheduled jobs** (no HTTP request): Use a `TenantIteratingJob` base class that loads active tenant IDs from the database (a server-controlled source) and processes each tenant in a `setTenantId()` → `execute` → `clear()` loop. This is the job-world equivalent of the JWT-only trust rule: the tenant identity comes from the database, not from any client input.
3. **Domain event listeners** running on the same thread inherit `TenantContext` automatically. Async event listeners use the same `TaskDecorator` mechanism as `@Async`.
4. `TenantContext.clear()` resets the set-once lock, enabling the per-iteration pattern.

**Consequences:** All execution contexts (HTTP, async, scheduled) have a defined tenant propagation story. No execution path can operate on tenant-scoped data without an explicitly set `TenantContext`. The trust source for each context is:

| Context                     | Tenant Source                       | Trust Level                 |
|-----------------------------|-------------------------------------|-----------------------------|
| HTTP request                | JWT <code>tenant_id</code> claim    | Cryptographically verified  |
| <code>@Async</code> task    | Propagated from calling HTTP thread | Inherited verified identity |
| <code>@Scheduled</code> job | Database <code>tenants</code> table | Server-controlled           |

See [Blueprint Critical Fixes](file:///C:/Users/Hajar/.gemini/antigravity-ide/brain/9024f939-3693-4a5c-93c1-6cc32aabd9bc/blueprint\_critical\_fixes.md) for full `TenantAwareTaskDecorator`, `TenantIteratingJob`, and `AsyncConfig` implementation.

## ADR-009: Entitlement-Based Feature Gating in Subscription Context

**Status:** Accepted **Context:** The `subscription/` bounded context defines Plans and billing but has no mechanism to enforce which features each plan unlocks. Without this, all tenants can access all features regardless of their subscription tier. **Decision:**

1. Introduce `Feature` (enum of gatable capabilities: `GUEST_IMPORT`, `SEATING_PLAN`, `HYBRID_PRINT`, `SMS_CHANNEL`, `WHATSAPP_CHANNEL`, `CUSTOM_BRANDING`, `ANALYTICS_ADVANCED`, `MULTI_EVENT`, `API_ACCESS`, `UNLIMITED_GUESTS`), `PlanFeature` (plan → feature mapping with optional limits), and `Entitlement` (runtime check result: entitled/not, remaining quota, reason).
2. `CheckEntitlementUseCase` resolves whether a tenant is entitled to a given feature based on their active subscription's plan.
3. `@RequiresEntitlement(Feature.X)` annotation provides declarative feature gating at the use-case or controller level, enforced by `EntitlementInterceptor` (Spring AOP).
4. `EntitlementChangedEvent` is emitted when a subscription upgrade/downgrade changes entitlements, allowing other contexts to react (e.g., disable seating plan UI).
5. Entitlement checks are cached per tenant (invalidated on plan change) to avoid per-request database lookups.

**Consequences:** Feature access is enforced at the application layer. New features can be gated by adding an enum value and annotating the relevant use case. Plan changes automatically reflect in entitlements via event-driven cache invalidation. The frontend can query `GetTenantEntitlementsUseCase` to hide/show UI features based on plan.

See [Blueprint Critical Fixes](file:///C:/Users/Hajar/.gemini/antigravity-ide/brain/9024f939-3693-4a5c-93c1-6cc32aabd9bc/blueprint\_critical\_fixes.md) for full `Feature`, `PlanFeature`, `Entitlement`, `@RequiresEntitlement`, and `EntitlementInterceptor` implementation.

---

## Reuse Summary (Updated)

- **Backend:** Hexagonal architecture within a modular monolith, organized by bounded context. Each context has explicit domain/application/infrastructure/presentation layers with enforced dependency rules. Java packages use lowercase concatenated words (no hyphens).
- **Frontend:** Standalone Angular structure with `core/`, `shared/`, `layout/`, and `features/` boundaries. Features are named by business domain, not generic placeholders. **All UI is built exclusively from OneUI 5.12 templates — no custom UI creation.**
- **UI foundation:** OneUI 5.12 (Bootstrap 5.3.8 admin template) provides all layouts, pages, blocks, forms, tables, charts, notifications, auth pages, error pages, dark mode, and RTL support. Angular components adapt these templates; they do not replace them.
- **Tenant support:** First-class architectural concern with dedicated `tenant/` context, discriminator column strategy, JWT-only trust rule (ADR-002-A), async propagation via `TenantAwareTaskDecorator` (ADR-008), and scheduled job propagation via `TenantIteratingJob` (ADR-008).
- **SaaS support:** Subscription/billing with **entitlement-based feature gating** (ADR-009), notifications, background jobs, file storage, and observability are integrated as dedicated packages with port/adapters.

boundaries.

- **Original conventions preserved:** `resource/` for controllers, `service/` for business logic, `repository/` for persistence — now placed within the correct architectural layer.